SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example. **Answer:** Testing should be treated as a systematic engineering activity rather than random testing. The team should follow a defined testing process with requirements analysis, test planning, test case design, execution, defect reporting, and regression testing. Techniques such as equivalence partitioning, boundary value analysis, decision table testing, and state transition testing can replace ad-hoc testing. Test management and automation tools can be used to maintain test cases and track defects. Standards, coding guidelines, review processes, and proper documentation should also be followed. For example, in an online banking application, different valid and invalid withdrawal amounts can be tested using equivalence classes and boundary values. This systematic approach improves defect detection, test coverage, reliability, and overall software quality.	3

2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes. **Answer:** A structured testing process should be followed throughout the SDLC. During requirements analysis, testable requirements are identified. During design, test scenarios and test conditions are prepared. During coding, unit testing is performed. During integration, modules such as card validation and account processing are tested together. During system testing, complete ATM operations are tested. Finally, acceptance testing confirms that the system meets user requirements. For ATM withdrawal, test cases should include valid withdrawal, insufficient balance, invalid PIN, maximum withdrawal limit, and invalid amount. A consistent process reduces missed defects and improves reliability. Expected outcomes include fewer failures, better coverage, and improved customer satisfaction.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples. **Answer:** An **error** is a human mistake made during development, such as writing an incorrect login condition. A **defect** is the fault in the software caused by that error, such as incorrect password validation. A **failure** occurs when the defect executes and produces an incorrect result, such as allowing an invalid password. The relationship is generally Error → Defect → Failure. Confusing these terms can make defect reporting unclear and delay debugging. Reports should clearly include defect description, steps to reproduce, expected result, actual result, severity, priority, and screenshots or logs. This makes communication between testers and developers more effective.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example. **Answer:** Defect clustering means that a small number of modules often contain most of the defects. The team should analyze previous defect data and identify high-risk modules. More test cases and testing effort should be assigned to these modules. Test cases can be prioritized based on risk, business importance, defect history, and complexity. Other testing principles such as exhaustive testing being impossible and testing showing the presence of defects should also guide planning. For example, in an e-commerce system, payment and checkout modules may have more defects than the product display module. Therefore, payment functions should receive higher testing priority. This improves testing efficiency and helps detect critical defects early.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements. **Answer:** Defects can originate from incorrect requirements, design mistakes, coding errors, integration problems, or environmental issues. They can be classified by type, severity, priority, module, root cause, and status. The defect repository should store complete information about each defect, including its cause, affected module, resolution, and test case. Testers can analyze historical defects to identify recurring patterns and create additional regression tests. Root-cause analysis, code reviews, improved requirements, and automated regression testing can prevent repeated defects. Proper use of the repository improves defect prevention, software reliability, and overall quality.	3

6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures. **Answer:** The tester should first reproduce the crash and collect logs, inputs, device information, and execution conditions. Tests should cover valid transfers, insufficient balance, invalid account numbers, network interruption, large amounts, concurrent transfers, and transaction timeouts. Boundary value, negative testing, integration testing, and stress testing can identify the root cause. Defects should be documented and analyzed using root-cause analysis. Automated regression tests should be added after fixing the issue. These practices reduce repeated failures and improve the reliability of the banking application.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively. **Answer:** Testing is the process of finding defects by executing and evaluating software, while debugging is the process of locating the cause of a defect and correcting the code. Testers mainly identify and report failures, whereas developers generally debug and fix the underlying causes. If the two are confused, defects may not be properly reported or investigated. A structured approach is to execute tests, record failures, report defects clearly, reproduce the problem, perform debugging, apply the fix, and conduct retesting and regression testing. This integration improves defect resolution and software quality.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

	Answer: Cross-browser testing should verify the application on different browsers, browser versions, operating systems, and screen sizes. Defects may originate from differences in HTML/CSS rendering, JavaScript support, browser engines, or responsive design. A test matrix should be prepared based on the browsers most commonly used by customers. Functional, compatibility, responsive, and regression testing should be performed. Tools such as Selenium and browser testing platforms can automate tests across multiple environments. Systematic cross-browser testing improves compatibility, consistency, and reliability.	
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy. **Answer:** Incorrect salary calculations caused by faulty program logic can be classified as logical or coding defects. Other classifications may include calculation, boundary, requirements, or data-related defects depending on the cause. Understanding defect classes helps testers select appropriate test techniques. For payroll, test cases should cover basic salary, deductions, overtime, bonuses, tax calculations, minimum and maximum values, and boundary conditions. Code reviews, unit testing, boundary value analysis, equivalence partitioning, and regression testing can prevent such defects in future releases.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness. **Answer:** Testing should begin with requirement analysis and test planning rather than random execution. Test cases should have a unique ID, objective, preconditions, inputs, steps, expected result, and actual result. Techniques such as equivalence partitioning, boundary value analysis, decision tables, state transition testing, and use-case testing should be used. For example, an age field can be tested using valid, invalid, minimum, and maximum values	3

	instead of random inputs. Structured test design improves coverage, repeatability, defect detection, and testing effectiveness.	
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting. **Answer:** Data inconsistency may occur because of incorrect database operations, improper data mapping, synchronization problems, or integration defects. The tester should verify data flow between modules and compare expected and actual database values. Integration testing, database testing, data validation, consistency checks, and regression testing should be performed. Defects should be recorded with module details, test data, expected result, actual result, severity, and reproduction steps. Systematic testing ensures that data remains accurate and consistent across the application.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects. **Answer:** The defect repository should be analyzed to identify repeated defects by module, type, severity, root cause, and release. For example, if repeated defects occur in payment processing, additional test cases should be created for that module. Historical defect information can guide risk-based testing and regression test selection. Root-cause analysis, code reviews, improved requirements, automated regression tests, and developer training can reduce recurring defects. Using repository data effectively helps prevent repeated failures and improves software quality.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3

	Answer: The failure indicates insufficient performance and load testing. Test planning should identify peak user loads, concurrent users, response-time requirements, and resource usage. Load, stress, endurance, and scalability testing should be included. For example, if the learning system normally supports 1,000 users but may receive 5,000 users during examinations, it should be tested under the expected peak load and beyond it. Monitoring CPU, memory, database performance, and response time helps identify bottlenecks. Better performance coverage improves reliability and prevents failures during peak usage.	
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices. **Answer:** Testers should participate from the requirements and design stages rather than joining only after coding. During requirements analysis, testers can identify ambiguous and non-testable requirements. During design, they can review workflows and prepare test scenarios. For example, in a banking application, early tester involvement can identify missing requirements for transaction limits and failed transactions. Developers, testers, business analysts, and stakeholders should collaborate through reviews and regular discussions. Early involvement helps detect defects sooner, reduces correction costs, and improves product quality.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation. **Answer:** Incomplete test coverage means that some requirements, conditions, paths, or input combinations have not been tested. The team should map requirements to test cases and identify uncovered areas. Equivalence partitioning, boundary value analysis, decision tables, state transition testing, and risk-based testing can improve coverage. For example, a banking transaction should be tested for valid amounts, zero, negative values, maximum limits, insufficient balance, and network failures. Proper	3

	coverage increases confidence that the system behaves correctly under different conditions.	
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards. **Answer:** Defect reports should clearly describe the problem so that developers can reproduce and fix it quickly. Each report should contain defect ID, title, module, environment, preconditions, steps to reproduce, expected result, actual result, severity, priority, and supporting evidence. Clear terminology helps distinguish errors, defects, and failures. Structured reporting reduces misunderstandings and improves communication between testers and developers. Standard templates and defect management tools should be used to improve reporting consistency.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency. **Answer:** A standardized testing process should be applied to all modules. The process can include requirement analysis, test planning, test case design, execution, defect reporting, retesting, and regression testing. Common test case templates, naming conventions, severity definitions, review procedures, and reporting standards should be used. For example, every module in an online shopping system can follow the same test case format and defect reporting process. Regular reviews, checklists, training, and automation can maintain consistency and improve quality.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3

	Answer: Edge cases may be missed when testing focuses only on normal inputs. Boundary Value Analysis helps identify errors at the minimum, maximum, just below, and just above valid limits. For example, if an account transfer limit is ₹50,000, test values such as ₹49,999, ₹50,000, and ₹50,001. Negative testing and equivalence partitioning should also be used. Requirements should explicitly identify boundary conditions, and edge-case tests should be included in regression suites. This reduces production failures.	
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices. **Answer:** A centralized defect repository should be maintained with complete information about every defect. Important fields include defect ID, description, module, severity, priority, environment, steps to reproduce, expected result, actual result, status, root cause, and resolution. Proper documentation makes it easier to track defects throughout their lifecycle. Repository data can also be analyzed to identify frequently affected modules and recurring defect types. Standard templates, regular updates, proper categorization, and periodic reviews improve defect management.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection. **Answer:** The banking system should adopt a structured testing process beginning with requirement analysis and risk-based test planning. Test cases should cover successful transactions, insufficient balance, invalid account details, transaction limits, duplicate transactions, network failures, timeouts, and concurrent transactions. Equivalence partitioning, boundary value analysis, decision tables, state transition testing, integration testing, and regression testing should be used. Automation can execute repetitive transaction tests, while defect repositories can help identify recurring	3

	problems. This structured approach improves coverage, detects defects earlier, reduces repeated failures, and increases transaction reliability.	

Assessment Tool Weightage: 50%

Code of Conduct:

I G Hemachandran 192421018 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Hemachandran

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand